

When Does an Embodied Agent Actually Self-Evolve? Causal Identification of Persistent Memory Across Episodes

Anonymous Authors

Advisor candidate – 5 September 2026

Preconfirmatory Advisor Draft. This manuscript separates current carrier evidence from a prospective longitudinal test. It does not claim that the confirmatory persistent-memory effect, or superiority of a governance policy, has already been observed. The base empirical scope is MemoryVLA/LIBERO-Plus.

Abstract

Embodied agents are increasingly described as self-evolving when they store experience and reuse it later. Yet memory-conditioned behavior within one episode does not establish persistent self-evolution: the update must survive across episodes, be retrieved under a native rule, and causally improve a separately frozen future episode relative to the exact pre-update persistent state. We propose a longitudinal identification standard built around a branch-isolated persistent-state fork: COMMITTED-UPDATE versus FROZEN-PREUPDATE. Source, verification, and future episodes are disjoint; the committed memory object has exact identity; native retrieval failures remain in the total effect rather than being silently dropped. Current evidence establishes an edit-capable carrier but not the prospective longitudinal claim. The planned confirmatory study tests semantic validity, persistent reuse, and the transport chain from insertion to future utility, with optional governance-method comparison only if an always-write baseline is prospectively executed.

1 Problem and premise

A one-step action change after memory exposure is not sufficient evidence that an embodied agent has learned persistently. Persistent self-evolution requires a lifecycle:

source experience $\rightarrow$ candidate memory $\rightarrow$ verification/admission $\rightarrow$ commit $\rightarrow$ future retrieval $\rightarrow$ closed-loop benefit.

The paper asks whether a prospectively governed persistent update, relative to the matched frozen pre-update state, causes a durable benefit on a separately frozen future episode.

The practical premise is stronger than generic “memory helps”: deployed embodied agents accumulate experience over time, and an update that helps only when manually injected or only within the acquisition episode is not a persistent learning result. Conversely, a persistent store can also accumulate stale, wrong, or irrelevant content. The scientific burden is therefore to identify the incremental effect of a particular committed update without changing the agent’s entire memory history.

2 Closest work and defended residual

Agentic Memory / AgeMem already studies when to store, retrieve, update, summarize, and discard long/short-term memory. Robo-Cortex and related embodied self-evolution systems already motivate long-term principle memory and autonomous knowledge induction. MemoryVLA and other VLA-memory systems already show that historical context can affect control. We therefore do not claim novelty for generic memory management, future reuse, or the phrase “self-evolving embodied agent.”

The defended residual is an identification standard:

exact persistent-state fork (COMMITTED-UPDATE vs. FROZEN-PREUPDATE) $\times$ disjoint source/verification/future roles $\times$ exact committed-object identity $\times$ native retrieval transport counted in the total effect $\times$ branch-isolated stream-level inference.

3 Scientific unit

One unit is a prospectively frozen source-experience/future-related-episode stream with one exact pre-update persistent-memory checkpoint and one candidate update generated from the source episode. The source episode creates the candidate; a separate verification episode decides whether the frozen admission rule accepts it; a distinct future episode supplies the primary endpoint.

The primary comparison forks from the same pre-update persistent state. This prevents the no-memory baseline from becoming the main causal comparator, because deleting all prior memory changes a much larger scientific object.

4 Current evidence and what is not yet known

Layer	Current evidence	Open confirmatory claim
Carrier	Existing action-shift evidence confirms that the embodied memory interface can alter policy behavior.	Carrier sensitivity is not persistent self-evolution.
Persistence	Candidate/pre-update states can be represented and forked in the planned harness.	A committed update must survive across episodes and be natively retrieved.
Semantics	Success/failure/provenance contrasts are available as controlled memory semantics.	Same-text semantic controls must separate useful content from added context/labels.
Utility	Not authorized by current carrier evidence.	Future native closed-loop benefit must differ under committed vs frozen pre-update state.

5 Prospective confirmatory design

5.1 Source / verification / future split

All roles are frozen before confirmatory execution. Source produces the candidate memory. Verification applies a prospectively specified admission rule without reusing the future endpoint. Future provides the primary outcome. Branches share the same initial persistent state and environment condition except for whether the candidate becomes a committed update.

5.2 Primary causal arms

Committed-Update: the verified candidate is written to the persistent store with exact object identity.

Frozen-Preupdate: the store remains at the exact pre-update checkpoint.

Secondary diagnostic controls may include **No-Memory** and a prespecified **Wrong/Unrelated Update** or token-matched sham. They do not replace the primary state fork.

If the paper later claims that the governance policy itself is superior, a third primary-method arm is required: **Always-Write** on the same candidate pool and future streams. Without it, the paper may claim a causal effect of a verified committed update, not superiority of the admission rule.

5.3 Native retrieval as part of the total effect

Future-task relevance and retrieval eligibility are frozen prospectively. If the committed memory is not retrieved under the native rule, that stream remains in the primary total-effect analysis as a transport failure. Manual injection is allowed only as a mechanism diagnostic, not as a rescue of the longitudinal claim.

5.4 Fast-loop mechanism readouts

Inside the future episode, mechanism diagnostics trace memory retrieval/exposure, first executable action change, downstream retention, and terminal outcome. These readouts can localize a transport bottleneck but cannot substitute for the primary closed-loop endpoint.

6 Minimum workload

A minimal confirmatory program has two stages. Phase A validates semantic sensitivity over 32 units under four matched memory conditions: no memory, success memory, failure memory, and a content-preserving flipped-label/provenance control. Phase B tests longitudinal persistence over 24 source–future streams with committed update, frozen pre-update, and a prespecified sham/no-consolidation diagnostic as appropriate. The planned base total is 200 runs. Expansion is not automatic; the paper can be split into a narrower causal-identification result and a later governance-method paper if the combined object becomes too broad.

7 Baselines and hard stops

The strongest causal baseline is the exact frozen pre-update state. NullMemory is useful for dependence on prior memory but changes the entire historical state. Wrong/stale memory tests extra-context explanations. Always-Write is mandatory only for a governance-superiority claim.

We stop or narrow if: static semantic distinctions fail; the committed update does not outperform the frozen pre-update branch; native retrieval transport collapses; gains disappear under same-text/different-provenance controls; or apparent benefit can be explained by merely adding persistent context. A failed primary effect is not rescued by opening more VLA backbones.

8 Relationship to Paper A

Paper A audits whether a memory exposure causes content-faithful policy use. Paper B asks a different longitudinal question: whether a governed persistent update survives across episodes and improves a later task. Paper-A-style influence/fidelity measures are therefore mechanism readouts inside Paper B, not Paper B’s primary claim. If the causal objects cannot be kept distinct under the final evidence, the two should be merged rather than protected by wording alone.

9 Scope and limitations

The first decisive claim is MemoryVLA/LIBERO-Plus bounded. It does not establish generic multi-VLA self-evolution, long-horizon deployment safety, or superiority of governance over always-write unless those gates are executed. One source/future substrate and a compact number of task families establish controlled identification, not deployment prevalence.

10 Conclusion

Persistent self-evolution should mean more than an agent changing after seeing memory. The proposed test asks whether one prospectively governed update, forked from the same persistent checkpoint, survives native retrieval and causes later closed-loop benefit. Current evidence supports the carrier and motivates the design; the longitudinal outcome remains prospective. The resulting paper is intentionally falsifiable: transport failure, semantic collapse, or lack of future benefit narrows the claim rather than triggering uncontrolled experiment expansion.

Related systems used to define the boundary

MemoryVLA and related VLA-memory systems establish embodied history use. Agentic Memory / AgeMem (ACL 2026) addresses learned memory operations, and Robo-Cortex (2026) motivates self-evolving embodied agents with long-term memory. Exact bibliographic metadata will be normalized before venue submission; these systems are cited here only to make the novelty boundary explicit.